thisdogdoesnotexist.com

1st Kuchar
Computer Science
Texas Christian University
Fort Worth, United States
griffin.kuchar@tcu.edu

2nd Shirk
Data Science
Texas Christian University
Fort Worth, United States
adam.john.shirk@tcu.edu

3rd Rodriguez
Computer Science
Texas Christian University
Fort Worth, United States
alfonso.tomax@tcu.edu

***Abstract*—This project includes the development of a Generative Adversarial Network to create realistic, artificial images of dogs. Inference will be served and accessible through a simple web UI.**

I. Problem Statement

The goal is unchanged from the proposal. We generate realistic 64×64 RGB images of dogs, compare a VAE baseline [1] against a GAN [2] using FID [3]. Dogs are harder than faces because poses, framing and breed vary far more. The main change is to our dataset which is now fixed in detail (Table I).

II. Dataset

To reach a sufficient amount of data, we will collate three unique data sets into one. All of the three data sets are free and available right now.

Dataset	**Images**
https://cg.cs.tsinghua.edu.cn/ThuDogs/	70,428
https://www.robots.ox.ac.uk/~vgg/data/pets/	4,978
https://data.mendeley.com/datasets/v5j6m8dzhv/1	1,657
Total	**77,063**

A. Data Cleaning

After cleaning, 76,287 images remain from three public datasets. They are split 95/2.5/2.5 into train, validation, and test (Table I). The 8x8 average-hash deduplication is run across the whole set before the split, so no near-duplicate can appear in both train and test.

TABLE I
Changes from the Approved Proposal

Area	**Proposal**	**Now**
Dataset	77,063 images	76,287 after removing grayscale, too-small, unannotated and duplicate images
FID reference	Generated set vs. training set	Per epoch: 5,000 samples vs. val. Final: 10,000 samples vs. train+val and vs. test
Training	100–200 epochs	25 epochs per model for the midterm, the same budget for both models

B. Preprocessing

1) *Crop:* each image is cropped to a square around the dog with 10% padding. THU-Dogs uses the largest annotated box (the body box). Oxford uses the bounding box of the pet pixels in its trimap. Mendeley has no annotations, so it uses a centered square crop.
2) *Resize:* to 64×64 with Lanczos resampling.
3) *Filter:* 315 images are removed: 300 grayscale images (mean channel difference under 2), 11 crops smaller than 48 px, and 4 THU-Dogs images with no annotation file.
4) *Deduplicate:* an 8×8 average hash removes 477 images with an identical hash.
5) *Store:* one uint8 array ($76{,}287 \times 64 \times 64 \times 3$, 937 MB) plus a metadata CSV with each image's source, path and split. Each batch is scaled to $[-1, 1]$ on the GPU during training.

III. Model Architectures

A. Baseline: Convolutional VAE

1) General Architecture: The VAE is composed of two major parts: Encoder and Decoder. The encoder takes a $3 \times 64 \times 64$ image scaled to $[-1, 1]$ and passes it through four convolutional blocks, each a 4×4 convolution with stride 2, batch normalization and a LeakyReLU (slope 0.2), with each block halving the spatial size and increasing the channels ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$), compressing the image into a $256 \times 4 \times 4$ feature map (0.69 M parameters). This map is flattened to 4,096 values and fed to two separate linear layers (1.05 M parameters together), which output the mean μ and log-variance $\log \sigma^2$ of a 128-dimensional Gaussian. Rather than one fixed code, each image is described as a region of the latent space. The reparameterization trick then samples a latent vector $z = \mu + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, which keeps the sampling step differentiable so gradients reach the encoder. The decoder reverses the process: a linear layer (0.53 M parameters) expands z to 4,096 values, which are reshaped to $256 \times 4 \times 4$. Four transposed convolutions (4×4, stride 2) then double the spatial size each step back to 64×64, reducing the channels

$128 \rightarrow 64 \rightarrow 32 \rightarrow 3$ (0.69 M parameters). The first three use batch normalization and ReLU, and the last uses Tanh so the output pixels lie in $[-1, 1]$, like the input. The KL term in the loss pulls every encoded region toward $\mathcal{N}(0, I)$. After training, the encoder is therefore discarded, and new dogs are generated by decoding random vectors $z \sim \mathcal{N}(0, I)$.

2) Loss and Training Updates: The VAE is trained by minimizing a loss with two terms: a reconstruction term, measured as the squared error (SE) between the input image x and the decoders output $\hat{x}$ summed over all $3\times64\times64$ pixel values. The second term is Kullback–Leibler (KL) a divergence term, measuring how far each image's distribution $\mathcal{N}(\mu, \sigma^2)$ lies from the standard normal prior $\mathcal{N}(0, I)$. The two terms are combined as $\text{SE}+\beta\cdot\text{KL}$ and averaged over the batch. In each training update, a batch of 128 images is encoded into μ and $\log\sigma^2$, a latent vector $z = \mu + \sigma\epsilon$ is sampled, and the decoder reconstructs the images from z. The loss $\text{SE}+\beta\cdot\text{KL}$ is then computed, and a single backward pass updates the encoder and decoder together using Adam.

TABLE II
VAE ARCHITECTURE

Stage	Layers	Output
Encoder	$4\times$ [Conv 4×4, stride 2, BatchNorm, LeakyReLU 0.2], channels $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$	$256 \times 4 \times 4$
Latent	Flatten, then two Linear($4096 \rightarrow 128$) heads for μ and $\log\sigma^2$; $z = \mu + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$	128
Decoder	Linear($128 \rightarrow 4096$), reshape, $3\times$ [ConvTranspose 4×4, stride 2, BatchNorm, ReLU], channels $128 \rightarrow 64 \rightarrow 32$	$32 \times 32 \times 32$
Output	ConvTranspose 4×4, stride 2, Tanh (no BatchNorm)	$3 \times 64 \times 64$

B. Frontier: Convolutional GAN

1) General Architecture: Our GAN is composed of two CNNs: the Generator, and the Discriminator. Overall, our GAN has 6,344,196 parameters, with the Generator accounting for 3,577,667 parameters and the Discriminator for 2,766,529 parameters (Table III). The Generator takes a random latent vector of 100 values obtained from the normal distribution, and forward propagates through its network until it outputs a 3x64x64 RGB image. In each of its hidden layers, it applies a 4x4 convolution, batch normalization, and uses ReLU as its activation function. The output layer similarly applies a 4x4 convolution, but uses tanh as its activation function in order to ensure each pixel value of the generated image is between -1 and 1. The Discriminator takes a 3x64x64 RGB image from either the training set or the Generator as input and outputs a binary classification of whether the input image is from the train set or from the Generator. In each of its hidden layers, it applies a 4x4 convolution, batch normalization (except in the first), and uses leaky ReLU as its activation function. The output layer applies a 4x4 convolution and collapses to a single neuron value. This value has sigmoid applied downstream to make its final prediction.

TABLE III
GAN ARCHITECTURE

Stage	Layers	Output
Generator input	Random latent vector $z \sim \mathcal{N}(0, I)$, reshaped to $100 \times 1 \times 1$	$100 \times 1 \times 1$
Generator	ConvTranspose 4×4, stride 1, pad 0, BatchNorm, ReLU (channels $100 \rightarrow 512$), then $3\times$ [ConvTranspose 4×4, stride 2, BatchNorm, ReLU], channels $512 \rightarrow 256 \rightarrow 128 \rightarrow 64$	$64 \times 32 \times 32$
Generator output	ConvTranspose 4×4, stride 2, Tanh (no BatchNorm), channels $64 \rightarrow 3$	$3 \times 64 \times 64$
Discriminator	Conv 4×4, stride 2, LeakyReLU 0.2 (no BatchNorm, channels $3 \rightarrow 64$), then $3\times$ [Conv 4×4, stride 2, BatchNorm, LeakyReLU 0.2], channels $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$	$512 \times 4 \times 4$
Discriminator output	Conv 4×4, stride 1, pad 0, collapses to one logit; sigmoid applied downstream	1

2) Loss and Training Updates: Both the Generator and the Discriminator use Binary Cross Entropy Loss (BCE). The Discriminator updates first: it scores a batch of real images against the target label 1 and a batch of generated images against the target label 0, and its loss is the sum of these two terms, which it minimizes by learning to separate real from fake. The Generator then updates on those same generated images, but runs them through the Discriminator with the target label 1, so its loss measures how far the Discriminator was from being fooled. This flipped label is the adversarial mechanism: the Discriminator is penalized for calling fakes real while the Generator is penalized for failing to make them look real, so the two networks push on the same images from opposite directions.

IV. FIRST EXPERIMENT QUANTITATIVE RESULTS

We evaluate success of the models by using The Fréchet Inception Distance. To document the on-going performance of the models, after each epoch we recorded the FID of a sample of 5000 generated images from each model against the validation set. For the final evaluation, we took a sample of 10000 generated images from each model, and calculate the Fréchet Inception Distance from each model's generated set to both the train + validation set and the test set. The GAN achieves lower FID than the VAE on both reference sets, consistent with GANs producing sharper samples than VAEs.

A. Baseline: Convolutional VAE

1) FIDs Against Validation Across Epochs: Figure 1 shows FID versus epoch for the VAE, measured against the validation set using 5,000 generated samples.

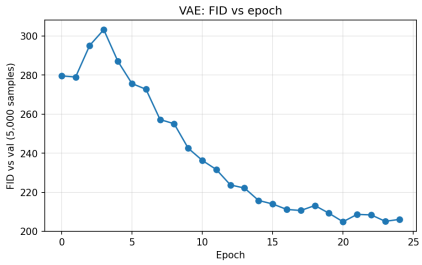


Fig. 1. FID versus epoch for the VAE, measured against the validation set using 5,000 generated samples.

2) Final FIDs: Table IV reports the final FID of the trained VAE, computed with 10,000 generated samples against each reference set.

TABLE IV
VAE FINAL FID BY REFERENCE SET

Reference set	**FID**
Train + Validation	194.88
Test	204.22

3) Example of The Final VAE's Image Generation: Figure 2 shows generated sample from the VAE at epoch 25.

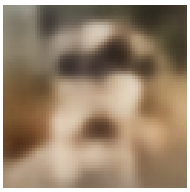


Fig. 2. Generated sample from the VAE at epoch 25.

B. Frontier: Convolutional GAN

1) FIDs Against Validation Across Epochs: Figure 3 shows FID versus epoch for the GAN, measured against the validation set using 5,000 generated samples.

2) Final FIDs: Table V reports the final FID of the trained Generator, computed with 10,000 generated samples against each reference set.

TABLE V
GAN FINAL FID BY REFERENCE SET

Reference set	**FID**
Train + Validation	118.95
Test	130.17

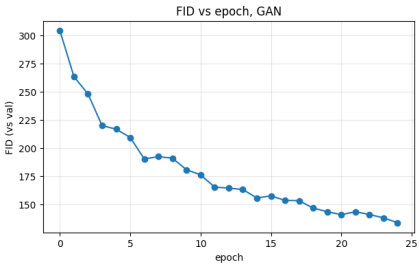


Fig. 3. FID versus epoch for the GAN, measured against the validation set using 5,000 generated samples.

3) Example of The Final GAN's Image Generation: Figure 4 shows generated sample from the GAN at epoch 25.

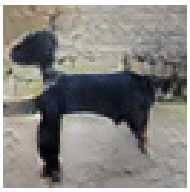


Fig. 4. Generated sample from the GAN at epoch 25.

V. CURRENT DIFFICULTIES & LIMITATIONS

One difficulty was running out of memory on Google Colab, which designates 12 GB of RAM. The dataset was stored as an unsigned int (uint8), which needed to be converted to a float32 datatype to scale to $[0, 1]$ in the VAE model. Converting the whole dataset from uint8 to float32 caused the session to crash. A workaround was keeping the data as uint8 and converting it to float32 for each batch. Another difficulty was Google Colab's session time limit and its occasional disconnections. Fortunately, training for 25 epochs took about 40 minutes, well under the session limit, and I was not disconnected from Colab during training. The session limit and the random disconnects may result in the loss of training, wasting our time. To combat this we saved checkpoints to drive on every epoch so if we disconnect we can resume from our save point.

VI. PLAN FOR REMAINING WORK

1) Extend training toward 100–200 epochs from Drive checkpoints.
2) Build a simple web interface that serves one dog generated per visit.
3) Write the final report with the full comparison.

VII. CODE REPOSITORY

The code is available at https://github.com/Alfonsotomax/Adfonfin.

REFERENCES

[1] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. 2nd Int. Conf. Learn. Representations (ICLR)*, 2014.

[2] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 27, 2014, pp. 2672–2680.

[3] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.